Gemini Chat

Here I ask Gemini 3 Pro to make the optimization simulator/ visualization. It one-shots the HTML page with no visual errors, and the interface is very clear and intuitive.

I want to create an interactive educational tool for my D
Please write a self-contained, single-file HTML/Javascr
The Environment: Render a 2D Contour Plot of a 'tricky'
The Agents: Implement three 'agents' starting at the sa
- SGD (Red dot/line)
- Momentum (Blue dot/line)
- Adam (Green dot/line)
The Controls: Sidebar controls with sliders for:
- Learning Rate (α)
- Momentum (β or γ)
- Adam parameters (β_1, β_2)
Animation: A 'Start/Reset' button. When clicked, the ag
Technical Constraints:
- Do not use external CSS/JS files; embed everything in
- Show a live legend updating the current (x, y) and l

Here is a complete, single-file HTML application. It uses **Plotly.js** (via CDN) for the rendering engine, as it handles contour p

I have selected the **Rosenbrock Function** (often called the "Banana Function") as the environment. It is the canonical "trick here (bouncing off the valley walls), while Momentum and Adam navigate the curve much more efficiently.

dssd

Instructions

1. Save the code below as an `.html` file (e.g., `optimizer_viz.html`).
2. Open it in any modern web browser.

HTML

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Optimizer Visualization: SGD vs Momentum vs Adam</title>
    <script src="https://cdn.plot.ly/plotly-2.24.1.min.js"></script>
    <style>
        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            margin: 0;
            padding: 0;
            display: flex;
            height: 100vh;
            background-color: #f4f4f9;
            overflow: hidden;
        }

        /* Sidebar Control Panel */
        #controls {
            width: 300px;
            background-color: #2c3e50;
            color: white;
            padding: 20px;
            display: flex;
            flex-direction: column;
            overflow-y: auto;
```

```
            box-shadow: 2px 0 5px rgba(0,0,0,0.1);
            z-index: 10;
        }

        h2, h3 { margin-top: 0; font-weight: 600; }
        h3 { font-size: 1rem; margin-top: 15px; border-bottom: 1px solid #485f78; padding-bottom: 5px; }

        .control-group { margin-bottom: 15px; }
        label { display: flex; justify-content: space-between; font-size: 0.85rem; margin-bottom: 5px; }

        input[type=range] { width: 100%; cursor: pointer; }

        .btn-group { display: flex; gap: 10px; margin-top: 20px; }
        button {
            flex: 1;
            padding: 10px;
            border: none;
            border-radius: 4px;
            cursor: pointer;
            font-weight: bold;
            transition: background 0.2s;
        }
        #btn-start { background-color: #27ae60; color: white; }
        #btn-start:hover { background-color: #2ecc71; }
        #btn-reset { background-color: #c0392b; color: white; }
        #btn-reset:hover { background-color: #e74c3c; }

        /* Stats Panel */
        #stats {
            margin-top: auto;
            background: rgba(0,0,0,0.2);
            padding: 10px;
            border-radius: 4px;
            font-family: 'Courier New', monospace;
            font-size: 0.8rem;
        }
        .stat-line { display: flex; align-items: center; margin-bottom: 4px; }
        .dot { width: 10px; height: 10px; border-radius: 50%; margin-right: 8px; }

        /* Chart Area */
        #chart-container {
            flex-grow: 1;
            position: relative;
        }
    </style>
</head>
<body>

    <div id="controls">
        <h2>Optimizer Race</h2>
        <p style="font-size: 0.8rem; color: #bdc3c7;">
            Surface: Rosenbrock Function<br>
            Goal: Reach (1, 1)
        </p>

        <div class="control-group">
            <label>Learning Rate (&alpha;) <span id="val-lr">0.001</span></label>
            <input type="range" id="lr" min="0.0001" max="0.01" step="0.0001" value="0.002">
        </div>

        <h3>Momentum Agent</h3>
        <div class="control-group">
            <label>Beta (&gamma;) <span id="val-mom">0.9</span></label>
            <input type="range" id="momentum" min="0.0" max="0.99" step="0.01" value="0.9">
        </div>

        <h3>Adam Agent</h3>
        <div class="control-group">
            <label>Beta 1 <span id="val-b1">0.9</span></label>
            <input type="range" id="beta1" min="0.5" max="0.999" step="0.001" value="0.9">
```

```
        </div>
        <div class="control-group">
            <label>Beta 2 <span id="val-b2">0.999</span></label>
            <input type="range" id="beta2" min="0.5" max="0.9999" step="0.0001" value="0.999">
        </div>

        <div class="btn-group">
            <button id="btn-start" onclick="toggleAnimation()">Start</button>
            <button id="btn-reset" onclick="resetSimulation()">Reset</button>
        </div>

        <div id="stats">
            <div class="stat-line"><div class="dot" style="background:red;"></div>SGD: <span id="loss-sg
            <div class="stat-line"><div class="dot" style="background:blue;"></div>Momentum: <span id="l
            <div class="stat-line"><div class="dot" style="background:green;"></div>Adam: <span id="loss-
            <div style="margin-top:8px; border-top: 1px solid #555; padding-top:5px;">Iter: <span id="ite
        </div>
    </div>

    <div id="chart-container">
        <div id="plot" style="width: 100%; height: 100%;"></div>
    </div>

    <script>
        // --- 1. CONFIGURATION & STATE ---
        const CONFIG = {
            startX: -1.5,
            startY: 2.0,
            targetX: 1,
            targetY: 1,
            range: { x: [-2, 2], y: [-1, 3] },
            stepsPerFrame: 5, // Speed up animation by doing multiple updates per render
            epsilon: 1e-8
        };

        let isRunning = false;
        let animationId = null;
        let iteration = 0;

        // Optimizer States
        let agents = {
            sgd: { x: 0, y: 0, pathX: [], pathY: [], loss: 0 },
            mom: { x: 0, y: 0, vx: 0, vy: 0, pathX: [], pathY: [], loss: 0 },
            adam: { x: 0, y: 0, mX: 0, mY: 0, vX: 0, vY: 0, pathX: [], pathY: [], loss: 0 }
        };

        // --- 2. MATH FUNCTIONS (Rosenbrock) ---
        // f(x,y) = (1-x)^2 + 100(y-x^2)^2
        function getLoss(x, y) {
            return Math.pow(1 - x, 2) + 100 * Math.pow(y - x * x, 2);
        }

        function getGradients(x, y) {
            // df/dx = -2(1-x) - 400x(y-x^2)
            // df/dy = 200(y-x^2)
            const dx = -2 * (1 - x) - 400 * x * (y - x * x);
            const dy = 200 * (y - x * x);
            return { dx, dy };
        }

        // --- 3. UI UPDATES ---
        const ui = {
            lr: document.getElementById('lr'),
            mom: document.getElementById('momentum'),
            b1: document.getElementById('beta1'),
            b2: document.getElementById('beta2'),
            btn: document.getElementById('btn-start'),
            iter: document.getElementById('iter-count'),
            lossSGD: document.getElementById('loss-sgd'),
            lossMom: document.getElementById('loss-mom'),
```

There is a lot of HTML code, but I had GPT confirm that the implementation of the optimization algorithms is indeed correct. I later have the LLM display and explain the equations for educational purposes.

```
        lossAdam: document.getElementById('loss-adam')
    };

    // Update slider value displays
    document.querySelectorAll('input[type=range]').forEach(input => {
        input.addEventListener('input', (e) => {
            // Find sibling span and update text
            e.target.previousElementSibling.firstElementChild.innerText = e.target.value;
        });
    });

    // --- 4. OPTIMIZATION LOGIC ---
    function step() {
        const lr = parseFloat(ui.lr.value);
        const beta = parseFloat(ui.mom.value); // Momentum
        const b1 = parseFloat(ui.b1.value);
        const b2 = parseFloat(ui.b2.value);

        iteration++;

        // 1. SGD Update
        let g = getGradients(agents.sgd.x, agents.sgd.y);
        agents.sgd.x -= lr * g.dx;
        agents.sgd.y -= lr * g.dy;
        agents.sgd.pathX.push(agents.sgd.x);
        agents.sgd.pathY.push(agents.sgd.y);
        agents.sgd.loss = getLoss(agents.sgd.x, agents.sgd.y);

        // 2. Momentum Update
        g = getGradients(agents.mom.x, agents.mom.y);
        agents.mom.vx = beta * agents.mom.vx - lr * g.dx;
        agents.mom.vy = beta * agents.mom.vy - lr * g.dy;
        agents.mom.x += agents.mom.vx;
        agents.mom.y += agents.mom.vy;
        agents.mom.pathX.push(agents.mom.x);
        agents.mom.pathY.push(agents.mom.y);
        agents.mom.loss = getLoss(agents.mom.x, agents.mom.y);

        // 3. Adam Update
        g = getGradients(agents.adam.x, agents.adam.y);

        // First Moment
        agents.adam.mX = b1 * agents.adam.mX + (1 - b1) * g.dx;
        agents.adam.mY = b1 * agents.adam.mY + (1 - b1) * g.dy;

        // Second Moment
        agents.adam.vX = b2 * agents.adam.vX + (1 - b2) * (g.dx * g.dx);
        agents.adam.vY = b2 * agents.adam.vY + (1 - b2) * (g.dy * g.dy);

        // Bias Correction
        let mHatX = agents.adam.mX / (1 - Math.pow(b1, iteration));
        let mHatY = agents.adam.mY / (1 - Math.pow(b1, iteration));
        let vHatX = agents.adam.vX / (1 - Math.pow(b2, iteration));
        let vHatY = agents.adam.vY / (1 - Math.pow(b2, iteration));

        agents.adam.x -= lr * mHatX / (Math.sqrt(vHatX) + CONFIG.epsilon);
        agents.adam.y -= lr * mHatY / (Math.sqrt(vHatY) + CONFIG.epsilon);

        agents.adam.pathX.push(agents.adam.x);
        agents.adam.pathY.push(agents.adam.y);
        agents.adam.loss = getLoss(agents.adam.x, agents.adam.y);
    }

    function updateDisplay() {
        // Update Text Stats
        ui.iter.innerText = iteration;
        ui.lossSGD.innerText = agents.sgd.loss.toFixed(4);
        ui.lossMom.innerText = agents.mom.loss.toFixed(4);
        ui.lossAdam.innerText = agents.adam.loss.toFixed(4);
```

```
        // Update Plotly Traces (Indices 1, 2, 3 based on initial setup)
        Plotly.extendTraces('plot', {
            x: [[agents.sgd.x], [agents.mom.x], [agents.adam.x]],
            y: [[agents.sgd.y], [agents.mom.y], [agents.adam.y]]
        }, [1, 2, 3]);
    }

    function animate() {
        if (!isRunning) return;

        // Perform multiple math steps per visual frame for smoothness
        for(let i=0; i<CONFIG.stepsPerFrame; i++) {
            step();
        }

        updateDisplay();
        animationId = requestAnimationFrame(animate);
    }

    function toggleAnimation() {
        if (isRunning) {
            isRunning = false;
            cancelAnimationFrame(animationId);
            ui.btn.innerText = "Resume";
            ui.btn.style.backgroundColor = "#f39c12"; // Orange
        } else {
            isRunning = true;
            ui.btn.innerText = "Pause";
            ui.btn.style.backgroundColor = "#e67e22"; // Darker Orange
            animate();
        }
    }

    function resetSimulation() {
        isRunning = false;
        cancelAnimationFrame(animationId);
        ui.btn.innerText = "Start";
        ui.btn.style.backgroundColor = "#27ae60"; // Green
        iteration = 0;

        // Reset Agent States
        agents.sgd = { x: CONFIG.startX, y: CONFIG.startY, pathX: [CONFIG.startX], pathY: [CONFIG.sta
        agents.mom = { x: CONFIG.startX, y: CONFIG.startY, vx: 0, vy: 0, pathX: [CONFIG.startX], patl
        agents.adam = { x: CONFIG.startX, y: CONFIG.startY, mX: 0, mY: 0, vX: 0, vY: 0, pathX: [CONFI

        // Reset Plot
        initPlot();
        ui.iter.innerText = "0";
        ui.lossSGD.innerText = "0.00";
        ui.lossMom.innerText = "0.00";
        ui.lossAdam.innerText = "0.00";
    }

    // --- 5. INITIALIZATION ---
    function generateContourData() {
        let x = [], y = [], z = [];
        let n = 100; // Resolution

        // Create range
        let xStep = (CONFIG.range.x[1] - CONFIG.range.x[0]) / n;
        let yStep = (CONFIG.range.y[1] - CONFIG.range.y[0]) / n;

        for (let i = 0; i <= n; i++) x.push(CONFIG.range.x[0] + i * xStep);
        for (let i = 0; i <= n; i++) y.push(CONFIG.range.y[0] + i * yStep);

        // Calculate Z
        for (let j = 0; j < y.length; j++) {
            let row = [];
            for (let i = 0; i < x.length; i++) {
                // Log scale the loss for better visualization of the valley
```

```
                // Adding 1 before log to avoid log(0)
                row.push(Math.log(getLoss(x[i], y[j]) + 1));
            }
            z.push(row);
        }
        return { x, y, z };
    }

    function initPlot() {
        const data = generateContourData();

        // Trace 0: Contour Background
        const contourTrace = {
            z: data.z,
            x: data.x,
            y: data.y,
            type: 'contour',
            colorscale: 'Viridis',
            ncontours: 40,
            contours: { showlabels: false },
            hoverinfo: 'none', // Disable hover on contour to reduce noise
            opacity: 0.6
        };

        // Common trace style
        const traceStyle = {
            mode: 'lines+markers',
            type: 'scatter',
            marker: { size: 8 },
            line: { width: 3 }
        };

        // Trace 1: SGD
        const sgdTrace = {
            ...traceStyle,
            x: [CONFIG.startX],
            y: [CONFIG.startY],
            name: 'SGD',
            line: { color: 'red', width: 2 },
            marker: { color: 'red', size: 6 }
        };

        // Trace 2: Momentum
        const momTrace = {
            ...traceStyle,
            x: [CONFIG.startX],
            y: [CONFIG.startY],
            name: 'Momentum',
            line: { color: 'blue', width: 2 },
            marker: { color: 'blue', size: 6 }
        };

        // Trace 3: Adam
        const adamTrace = {
            ...traceStyle,
            x: [CONFIG.startX],
            y: [CONFIG.startY],
            name: 'Adam',
            line: { color: 'lime', width: 2 },
            marker: { color: 'lime', size: 6 }
        };

        // Trace 4: Global Minimum (Star)
        const targetTrace = {
            x: [1],
            y: [1],
            mode: 'markers',
            type: 'scatter',
            name: 'Global Min',
            marker: { symbol: 'star', color: 'gold', size: 15, line: {color: 'black', width: 1} }
```

```
            };

            const layout = {
                title: 'Loss Surface (Log Scale)',
                xaxis: { range: CONFIG.range.x, title: 'X' },
                yaxis: { range: CONFIG.range.y, title: 'Y' },
                showlegend: true,
                legend: { x: 0, y: 1, bgcolor: 'rgba(255,255,255,0.7)' },
                margin: { l: 50, r: 20, t: 40, b: 40 }
            };

            const config = { responsive: true };

            Plotly.newPlot('plot', [contourTrace, sgdTrace, momTrace, adamTrace, targetTrace], layout, c
        }

        // Boot up
        window.onload = () => {
            initPlot();
            resetSimulation();
        };

    </script>
</body>
</html>
```

Here I ask Gemini to improve the visualization by adding more analysis about how Adam, Momentum, and SGD behave with different parameters. This part isn't too impressive, as Gemini added very minimal information like how SGD diverges with too high of a learning rate, but doesn't go in depth.

To make this a better study tool, can you add a "status box" somewhere? It should display a detailed text explanat
beta combinations, etc.

I have updated the tool to include a dynamic **"Live Analysis"** console.

Key Changes & Features:

1. **Live Analysis Console:** A new panel in the sidebar that acts as a "tutor," explaining *why* things are happening in real-t
2. **Oscillation Detection:** The code now calculates if SGD is "zig-zagging" (sign of the gradient flipping repeatedly) and
3. **Valley Detection:** It detects when agents enter the "Rosenbrock Valley" (the flat parabolic area) and explains why Mor
4. **Parameter Context:** When you move the sliders (Learning Rate, Beta), the text box immediately explains the theoretic

Here is the complete, single-file solution:

Optimizer_Analysis_Tool.html
Dec 7, 6:07 PM

At this point, Gemini switches to Canvas mode rather than just outputting the HTML for easier interface access.

Can you also add information that I can click to explain more about th

I've updated the tool to include a **"Math & Theory"** guide.

I added a new button in the sidebar (purple) that opens an overlay. This overlay breaks down the exact formulas used by the intuition for why Adam uses two different betas.

Here is the updated code:

Can you be more detailed in the guide about why momentum beats SGD and why Adam usually beats momentum parameters to show that?

I have updated the "Math & Theory" guide with a much deeper dive. It now mathematically explains the cancellation effect i so you can replicate specific scenarios (like SGD oscillation vs. Adam stability) yourself.

Here is the updated file:

Interestingly, despite repeated prompting to tell Gemini to be detailed in its explanations in the visualization/simulation, its guide ends up being realtively short. The bullets are all short and the total text typically spans no longer than 2 paragraphs even when I tell it to be detailed. It even rewrote some of its old content even though I didn't say to.

Your math & theory section has a lot of math markdown/latex that isn't rendering on my side. Can you fix it? Also c them does?

I'll fix the rendering issues by converting the LaTeX syntax into standard HTML entities (like `&alpha;` for α and `&nabla;` fo include a deeper explanation of the Beta parameters and the practical effects of changing them.

At this point, while not visible in the chat, the model hallucinated the rendering capabilities of the environment. It treated the generated HTML file as if it were a Markdown viewer (like a Jupyter notebook or the Gemini chat window itself) that automatically parses LaTeX. In a standalone HTML file, raw LaTeX code just looks like gibberish to the browser unless a library like MathJax is explicitly included. The model failed to recognize the constraints of the "single-file HTML" request it was fulfilling.

Here is the updated Canvas file:

When the learning rate is stable (no divergence), it seems that momentum always

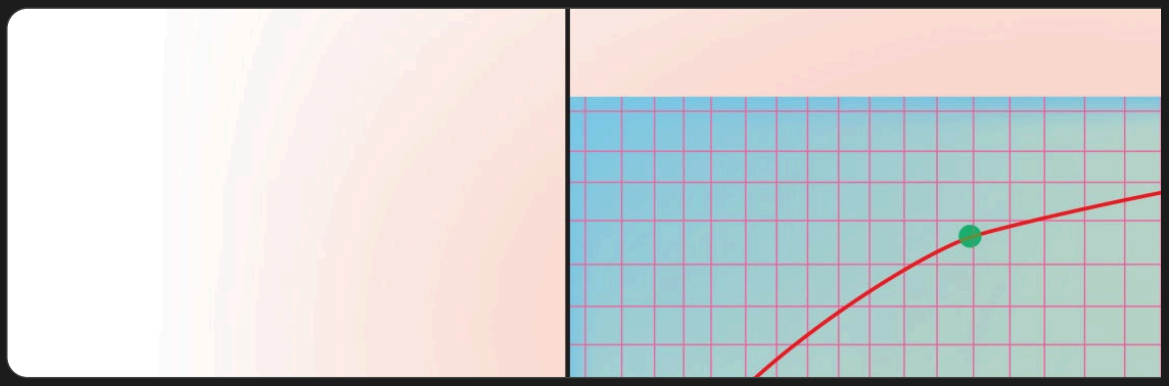
Yes, what you are observing is **completely expected behavior** for this specific type of problem (a deterministic, noise-free

In this "clean" simulation, **Momentum acts like a race car, while Adam acts like a heavily regulated safety vehicle.**

1. The "Speed Limit" Difference

The fundamental mathematical difference leads to this behavior:

- **Momentum (Velocity Accumulation):** Momentum adds the current gradient to a running total. If the gradient stays cc



```
Mathematically, the effective step size for Momentum can grow to approximately:
$$\text{Step} \approx \frac{\text{Gradient} \times \alpha}{1 - \beta}$$
If $\beta = 0.9$, Momentum multiplies the gradient's effect by **10x**. It allows the agent to build up r
```

- **Adam (Normalization/Clipping):** Adam divides the momentum (m_t) by the variance (v_t, essentially the magnitude of

$$\text{Step} \approx \alpha \cdot \frac{m_t}{\sqrt{v_t}} \approx \alpha \cdot \frac{\text{Gradi}}{\text{Gradi}}$$

Because Adam normalizes by the gradient magnitude, it effectively **cancels out** the size of the gradient. It forces the u

Result: Adam has a "speed limit" set by your Learning Rate. It cannot accelerate beyond that α limit, whereas Moment

2. Why use Adam then?

If Momentum is faster here, why is Adam the default for Deep Learning?

1. **Stochastic Gradients (Noise):** In real Deep Learning, you don't calculate the loss on the *entire* dataset at once (which jagged and noisy.
    - **Momentum** gets thrown off course easily by noise because it carries so much inertia.
    - **Adam** is incredibly stable in noise because it constantly re-scales itself based on variance (β_2).
2. **Vanishing Gradients:** In deep networks, gradients can become microscopically small (0.0000001).
    - **Momentum** would slow to a crawl (since step size is proportional to gradient).
    - **Adam** ignores the tiny magnitude (dividing small by small = 1) and keeps marching at speed α.

Summary

In a **smooth, deterministic valley** (like the Rosenbrock function in this tool), Momentum usually wins because it is allowed

Gemini's response in the chat is a lot more detailed than what it writes in the HTML. I had GPT fact check it, and GPT confirms that this is non-hallucinated insight that is true. I found it to be very helpful in clarifying my observations about the speed of Adam vs Momentum. I now realize that Adam's value is in its stability, not speed.